Name: Ankita Tank
Course: CSC 580
Assignment: Final Project
AI Tools Consulted: GitHub Copilot (Claude), PyTorch documentation, Dreamer V1 paper (Hafner et al., 2020)
Word Count: ~3900

1. Introduction

This report documents the design, implementation, and evaluation of Tiny Dreamer, a from-scratch Dreamer V1-style world model agent built in PyTorch for the highway-v0 environment from the Gymnasium Highway-Env suite. The agent's objective is to drive at high speed on a multi-lane highway while avoiding collisions with surrounding traffic all through latent imagination rather than brute-force model-free trial and error.

The Dreamer framework, introduced by Hafner et al. (2020), represents a paradigm shift in reinforcement learning. Instead of learning a policy through millions of direct environment interactions, Dreamer first learns a compact, differentiable world model that captures the environment's dynamics in a learned latent space. The agent's policy and value function are then trained entirely within this imagined world, drastically reducing the number of real environment interactions needed. This project implements the core principles of Dreamer V1 from scratch, adapting the architecture for the highway driving domain with continuous control.

2. Background and Motivation

Traditional model-free reinforcement learning algorithms (e.g., DQN, PPO, SAC) learn a policy by directly interacting with the environment, often requiring millions of steps to converge. This is particularly costly in visually rich environments where each observation is a high-dimensional image. Model-based reinforcement learning offers a compelling alternative: learn a dynamic model of the environment first, then plan or optimize a policy within that model.
Dreamer V1 (Hafner et al., 2020) is a landmark model-based RL algorithm that achieves competitive or superior performance to model-free methods while using significantly fewer environment interactions. Its key innovations include:

1. Latent Dynamics Learning: Rather than predicting future images directly (which is computationally expensive and prone to compounding errors), Dreamer learns dynamics in a compact latent space using a Recurrent State-Space Model (RSSM).
2. Imagination-Based Policy Optimization: The actor and critic are trained on imagined trajectories generated by the learned world model. Because the world model is differentiable, gradients can flow from future imagined rewards back through the dynamics to the actor, providing a rich credit-assignment signal.

3. Dual State Representation: The RSSM maintains both a deterministic component (GRU hidden state) for capturing predictable temporal patterns and a stochastic component (sampled from a learned Gaussian) for representing inherent environmental uncertainty.

The highway-v0 environment is an excellent testbed for this approach. It features structured visual scenes (lanes, cars, road markings), continuous dynamics (speed, position, lane changes), and clear reward signals (high speed is good, collisions are catastrophic). The visual simplicity relative to Atari games makes it tractable for a "Tiny Dreamer" implementation while still posing real challenges: the agent must anticipate the motion of other vehicles, plan lane changes, and balance speed against safety.

3. Environment Setup

The highway-v0 environment is configured with several deliberate choices that impact both training stability and the quality of the learned world model:

Parameter	Value	Rationale
render_mode	"rgb_array"	Provides pixel observations as NumPy arrays for the encoder
action type	ContinuousAction	Throttle + steering as two continuous floats in [-1, 1]
lanes_count	4	Richer driving scenarios with more lane-change opportunities
simulation_frequency	15 Hz	Physics simulation rate for smooth dynamics
policy_frequency	5 Hz	Agent decisions per second; prevents extreme spinning behavior
duration	40 seconds	Episode length (~200 steps at 5 Hz)
offroad_terminal	True	Terminates episodes when car leaves road
normalize_reward	True	Stabilizes reward scale for neural network training
collision_reward	-1.0	Strong negative signal for hitting other cars
high_speed_reward	0.4	Incentivizes maintaining highway cruising speed
right_lane_reward	0.1	Mild preference for right-lane driving
reward_speed_range	[20, 30]	Reward band aligned with typical highway speeds

A critical engineering decision was the relationship between simulation_frequency and policy_frequency. The default 1 Hz policy frequency causes a single steering action to persist for an entire second of simulation, resulting in the car completing near-full circles. By raising the policy

frequency to 5 Hz, each action only persists for 3 simulation steps (15/5), producing short, correctable turns that eliminate the spinning artifact.

Frames are preprocessed by resizing to 64×64 pixels using area interpolation (via OpenCV), converting from HWC to CHW format, and normalizing pixel values to [0, 1]. This compact representation is critical for making the CNN encoder computationally tractable while retaining sufficient visual detail to distinguish lanes, cars, and road boundaries.

4. System Architecture

The Tiny Dreamer system consists of six neural network components organized into two subsystems: the World Dynamics Model and the Actor-Critic Model.

4.1 Encoder (CNN)

The encoder compresses 64×64 RGB frames into a 256-dimensional embedding vector through a series of four convolutional layers:

Layer	Output Channels	Kernel	Stride	Output Size
Conv2d + ReLU	32	4×4	2	31×31
Conv2d + ReLU	64	4×4	2	14×14
Conv2d + ReLU	128	4×4	2	6×6
Conv2d + ReLU	256	4×4	2	2×2
Flatten + Linear				256

The progressive downsampling from 64×64 to 2×2 achieves a 768× spatial compression before the final linear projection. Each convolutional layer uses ReLU activations for non-linearity. The flattened feature map (256 × 2 × 2 = 1024 dimensions) is projected to the 256-dimensional embedding space through a fully connected layer.

The encoder's role is to extract a compact representation that captures the salient features of each frame vehicle positions, lane markings, and road geometry while discarding irrelevant details. This embedding becomes the observation that the RSSM conditions on during training.

4.2 RSSM Recurrent State-Space Model

The RSSM is the heart of the Dreamer world model. It maintains two complementary state representations:

- Deterministic state ($h_t \in R^{128}$): Computed by a GRU cell, this captures long-term temporal dependencies and deterministic dynamics (e.g., a car moving at constant speed follows a predictable trajectory).
- Stochastic state ($z_t \in R^{64}$): Sampled from a diagonal Gaussian distribution, this captures inherent environmental uncertainty (e.g., when will another car change lanes).

The RSSM maintains two distributions at each timestep:

1. Prior $p(z_t|h_t)$: Predicts the stochastic state from the deterministic state alone. Used during imagination when no real observation is available.

2. Posterior $q(z_t|h_t, e_t)$: Conditions on both the deterministic state and the actual observation embedding. Used during training to incorporate ground truth information.

The transition dynamics proceed as follows: 1. The previous stochastic state z_{t-1} and action a_{t-1} are concatenated and passed through a pre-GRU MLP (linear + ReLU). 2. The GRU cell updates the deterministic state: $h_t = GRU(MLP(z_{t-1}, a_{t-1}), h_{t-1})$. 3. The prior and posterior are computed from h_t (and e_t for the posterior) through separate MLP heads. 4. The stochastic state is sampled using the reparameterization trick for differentiable sampling.

During training, the KL divergence between prior and posterior is minimized, encouraging the prior to become an accurate predictor of the next state even without observations. This is what enables imagination: during imagined rollouts, only the prior is used.

4.3 Decoder (Transposed CNN)

The decoder mirrors the encoder architecture in reverse, reconstructing a 64×64 RGB image from the latent state:

Layer	Channels	Kernel	Stride	Output
Linear(192 → 1024)				Reshape to 256×2×2
ConvTranspose2d + ReLU	128	4×4	2	6×6
ConvTranspose2d + ReLU	64	4×4	2	14×14
ConvTranspose2d + ReLU	32	5×5	2	31×31
ConvTranspose2d + Sigmoid	3	6×6	2	64×64

The final Sigmoid activation constrains output pixel values to [0, 1]. The asymmetric kernel sizes (5×5 and 6×6 in the last two layers) are carefully chosen to match the exact spatial dimensions produced by the encoder's downsampling, ensuring the reconstructed image is exactly 64×64.

The decoder serves two purposes: (1) its reconstruction loss provides a learning signal for the encoder and RSSM, and (2) it enables visualization of what the world model "imagines," which is essential for evaluating prediction quality.

5. Training Procedure

5.1 Replay Buffer and Warm Start

The replay buffer stores up to 50,000 transitions as tuples of (frame, action, reward, next_frame, done). It supports sampling contiguous sequences of configurable length, which is essential because the RSSM is a recurrent model that needs temporal context. The sequence sampling carefully avoids crossing episode boundaries by checking that no done flag appears within the sampled window.

Training begins with a warm-start phase of 2,000 random exploration steps. A critical design choice here is the use of forward-biased random actions: acceleration is sampled from [0.1, 0.7] and steering from [-0.3, 0.3], rather than fully uniform random over [-1, 1]. Fully uniform random actions produce extreme steering commands that spin the car off-road, flooding the replay buffer with degenerate

trajectories that the world model cannot learn useful dynamics from. The biased distribution ensures warm-start data contains physically plausible highway driving behavior.

5.2 World Model Training

The world model is trained for 50 gradient steps per outer iteration. Each step samples a batch of 32 sequences of length 16 from the replay buffer. The loss function has three components, following Dreamer V1 (Equation 4):

$$L_{world} = L_{recon} + L_{KL} + L_{reward}$$

- Reconstruction Loss (L_{recon}): MSE between the decoded frames and the ground-truth frames. This trains the encoder-decoder pair and provides a learning signal for the latent representations.
- KL Divergence (L_{KL}): KL between the posterior $q(z_t|h_t, e_t)$ and prior $p(z_t|h_t)$. This is computed analytically for diagonal Gaussians. A free nats threshold of 1.0 is applied; the KL loss is clamped to a minimum of 1.0 nats per latent dimension to prevent the posterior from collapsing to a point estimate, which would eliminate the stochastic component entirely.
- Reward Loss (L_{reward}): MSE between predicted and actual rewards. This trains the reward model to accurately forecast rewards in latent space.

All four world model components (encoder, RSSM, decoder, reward model) share a single Adam optimizer with learning rate 3e-4. Gradient norms are clipped to 100.0 for stability.

5.3 Actor-Critic Training via Imagination

The actor-critic training is the distinctive innovation of Dreamer. It proceeds in two passes per gradient step:

Pass 1 Critic Update: Starting from real encoded observations, the RSSM imagines forward for 15 steps using the current actor's policy. The critic is trained to minimize MSE against TD-λ targets computed from imagined rewards:

$$V_t^{\lambda} = r_t + \gamma\left[(1 - \lambda)V(s_{t+1}) + \lambda V_{t+1}^{\lambda}\right]$$

The TD-λ formulation (with $\gamma = 0.99$ and $\lambda = 0.95$) provides a bias-variance tradeoff between pure Monte Carlo returns and one-step TD bootstrapping. All imagination is done with torch.no_grad() for the critic passing the critic's gradient only flows through its own network.

Pass 2 Actor Update: A fresh imagined trajectory is generated, this time with gradients flowing through the RSSM dynamics. This is the "straight-through" backpropagation approach from Dreamer V1: because the world model is differentiable, gradients from future imagined rewards propagate backward through the dynamics model to the actor's action outputs. This gives the actor a much richer credit-assignment signal than standard policy gradient methods, which only receive scalar returns.

5.4 Full Training Loop

The outer training loop runs for 100 iterations. Each iteration:

1. Trains the world model for 50 gradient steps on replay buffer sequences.
2. Trains the actor-critic for 30 gradient steps on imagined rollouts.
3. Collects 200 real environment steps using the learned policy and stores them in the replay buffer.

This interleaving ensures the world model continuously improves from new experience while the policy adapts to the latest dynamics model. Evaluation metrics are logged every 10 iterations, and prediction comparison videos are saved every 25 iterations.

6. Hyperparameter Configuration

Category	Parameter	Value
Latent Space	Stochastic dim (z)	64
	Deterministic dim (h)	128
	Encoder embedding dim	256
	MLP hidden dim	128
Training	Batch size	32
	Sequence length	16
	World model steps/iter	50
	Actor-critic steps/iter	30
	Total outer iterations	100
	Environment steps/iter	200
Optimization	World model LR	3e-4
	Actor LR	3e-4
	Critic LR	3e-4
	Max gradient norm	100.0
RL	Discount (γ)	0.99
	GAE lambda (λ)	0.95
	Imagination horizon	15 steps
	Entropy scale (α)	1e-3
Buffer	Capacity	50,000
	Warm-start steps	2,000

The latent dimensions (64 stochastic, 128 deterministic) were chosen to be large enough to represent the highway scene's relevant features (up to ~20 vehicle positions plus road geometry) while remaining small enough for tractable imagination. The sequence length of 16 balances the RSSM's need for temporal context against memory constraints. The imagination horizon of 15 steps represents approximately 3 seconds of driving at 5 Hz, which is sufficient for evaluating lane-change decisions without compounding prediction errors excessively.

7. Results and Analysis

7.1 World Model Loss Curves

The world model total loss (training_curves.png) shows the combined trajectory of the reconstruction, KL divergence, and reward prediction losses across all 100 training iterations.

Reconstruction loss is expected to decrease sharply during the first 20–30 iterations as the encoder-decoder pair learns the visual structure of the highway environment. The highway scene is relatively structured, lanes are parallel lines, cars are rectangular blobs, and the road occupies a consistent region of the frame so the reconstruction task is tractable even for a compact latent space. A well-converged reconstruction loss indicates that the latent representations contain sufficient information to reconstruct the visual scene, which is a necessary (but not sufficient) condition for accurate dynamics prediction.

7.2 Actor-Critic Loss Curves

The actor loss reflects the negative discounted return from imagined trajectories, so decreasing actor loss indicates improvement. In the early stages, the actor explores broadly due to the entropy bonus. As the world model improves and the imagined trajectories become more realistic, the actor receives more accurate reward signals and refines its policy toward high-speed, collision-free driving.

The critic loss (MSE against TD-λ targets) typically spikes initially as the critic adjusts to the changing world model, then stabilizes as both models converge. Persistent oscillations in the critical loss may indicate that the world model is still drifting, causing the distribution of imagined states to shift faster than the critic can track.

7.3 Episode Reward Progression

The episode reward plot (episode_rewards.png) tracks the actual return achieved by the agent in the real environment across training. This is the most direct measure of the system's overall effectiveness. During the warm start, episode rewards reflect random policy performance. As training progresses, we expect to see an upward trend as the policy learns to maintain high speed and avoid collisions. The smoothed reward curve (5-episode moving average) filters out the high variance inherent in individual episodes and reveals the underlying learning trajectory.

The reward signal in highway-v0 is a combination of speed reward (maximized around 20–30 m/s), collision penalty (-1), and right-lane preference (0.1). A successful agent should learn to predominantly earn the high-speed reward component while minimizing collision events. The normalized reward range means that per-step rewards are approximately [0, 1], so a good episode with 200 steps at near-maximum speed reward should yield total returns in the range of 30–60.

7.4 n-step Prediction Quality

The n-step prediction evaluation (evaluation_metrics.png, prediction_grid.png) is the most revealing assessment of the world model's quality. It measures how prediction mean squared error (MSE) grows as the model imagines further into the future from a single initial observation.

The prediction MSE vs. imagination step plot is expected to show a monotonically increasing trend. In the first few steps (1–5), predictions should closely match ground truth because the deterministic dynamics (car momentum, lane markings) dominate. Beyond 10–15 steps, compounding errors accumulate: small inaccuracies in predicted vehicle positions cascade through the RSSM, leading to progressively blurrier and less accurate reconstructions.

The prediction_grid.png visualization directly shows this degradation. Ground truth frames (top row) should remain sharp and detailed throughout, while predicted frames (bottom row) may become progressively blurrier or show incorrect vehicle positions at later timesteps. The rate of this degradation is a key quality metric: a well-trained world model maintains recognizable scene structure for at least 10–15 steps, while a poorly trained one produces unrecognizable outputs after just 3–5 steps.

7.5 Reward Prediction Accuracy

The reward scatter plot (evaluation_metrics.png, right panel) compares predicted vs. ground truth rewards across all evaluation steps. A perfect reward model would place all points on the diagonal line (y = x).

The reward prediction MAE quantifies the average absolute error. For the normalized highway-v0 rewards (roughly in [0, 1]), an MAE below 0.1 would indicate good reward prediction. The scatter plot will likely show tighter clustering around the diagonal for common reward values (cruising at highway speed) and more scatter for rare events (collisions, extreme decelerations).

The accuracy of the reward model is critical because the actor is trained to maximize *predicted* rewards during imagination. If the reward model overestimates rewards for dangerous situations (e.g., predicting positive reward when the car is about to collide), the actor will learn a reckless policy. Conversely, if it underestimates rewards for good driving, the actor will be overly conservative.

7.6 Dream Sequence Analysis

The dream sequence visualization (dream_sequence.mp4, dream_frames.png, dream_reward_trajectory.png) shows what happens when the world model runs in pure imagination mode for 50 steps from a single initial observation no real environment interaction occurs.

The dream frames reveal the world model's internal representation of highway driving. Early dream frames (steps 1–10) should show a recognizable highway scene with clear lane markings and vehicle positions. As the dream extends further, the model may exhibit characteristic behaviors:

1. Visual degradation: Frames become blurrier as uncertainty accumulates, reflecting the stochastic nature of the RSSM's prior distribution without posterior correction.

2. Plausible dynamics: Vehicles should move realistically (forward motion, lane maintenance) for at least the first 15–20 steps.
3. Reward consistency: The imagined reward trajectory should reflect plausible driving scenarios sustained positive rewards during highway cruising with occasional dips.

The dream reward trajectory plot shows the reward the model predicts at each imagination step. A well-trained model produces relatively stable positive rewards during imagined highway driving, reflecting the agent's expectation of continued successful driving.

7.7 Final Evaluation Episodes

The final evaluation runs 5 complete episodes with the trained policy in the real environment (agent_episode_*.mp4). The key metrics are:

1. Mean Reward: The average total reward across evaluation episodes. Higher is better.
2. Mean Steps: Average episode length. Longer episodes suggest the agent avoids collisions (which terminate the episode via offroad_terminal).
3. Standard Deviation: Low variance across episodes indicates consistent, reliable driving behavior.

These metrics provide the ground truth assessment of whether the world model and policy actually learned to drive effectively. Even if the world model's predictions are imperfect, the policy may still perform well in the real environment if it learned robust driving strategies from imagination.

8. Key Design Decisions and Engineering Challenges

Several engineering decisions were critical to making the system work:

1. Forward-biased random exploration. Fully uniform random actions ([-1, 1] on both throttle and steering) produce physically implausible behavior: the car spins wildly and leaves the road within a few steps. By biasing acceleration toward positive values [0.1, 0.7] and restricting steering to [-0.3, 0.3], the warm-start data contains forward-moving, on-road trajectories that the world model can learn plausible dynamics from.
2. Policy frequency tuning. Increasing the policy frequency from the default 1 Hz to 5 Hz was essential. At 1 Hz, a maximally steered car executes a full revolution in approximately one second. At 5 Hz, steering inputs are corrected every 200ms, producing smooth, controllable turns.
3. Off-road termination. Enabling offroad_terminal prevents the car from driving off-road indefinitely, which would generate degenerate transitions that confuse the world model. This aligns the termination signal with the driving task's natural boundaries.
4. Previous action tracking in inference. A subtle but important fix: during experience collection and evaluation, the RSSM needs to receive the *actual* previous action at each step, not a zero vector. The RSSM's transition dynamics are conditioned on the previous action, and feeding

zeros every step disconnects the recurrent state from the true action history, breaking the learned dynamics.

5. Separate imagination passes for actor and critic. Following Dreamer V1, the critic is trained on trajectories imagined *without* gradients through the actor, while the actor receives gradients through the full imagination chain. This separation prevents the critic's loss from interfering with the actor's gradient computation and ensures stable learning.

9. Limitations and Future Work

This implementation is intentionally a "Tiny" Dreamer; it trades scale for clarity and tractability. Several known limitations represent opportunities for future improvement:

Scale limitations. Our model uses relatively small latent dimensions (64 stochastic, 128 deterministic) and trains for only 100 iterations. The original Dreamer V1 used 30-dimensional stochastic states with 200 deterministic units and trained for millions of environmental steps. Scaling up would likely improve prediction accuracy and driving performance.

Image-based observation only at render time. The environment uses kinematic observations for step() and only renders images via env.render(). This means the agent doesn't receive pixel observations natively during stepping we render after each step. A tighter integration would use pixel observations directly as the environment's observation type.

Categorical vs. Gaussian latents. Dreamer V2 and V3 switched from Gaussian to categorical distributions for the stochastic state, which improved training stability and representation quality. Implementing categorical latents with straight-through gradients would be a natural next step.

Discount prediction. Dreamer V1 includes a discount predictor that learns the probability of episode continuation, enabling the model to account for terminal states during imagination. Our implementation uses a fixed discount factor, which means imagined trajectories don't account for potential episode termination.

Compounding prediction error. As shown in the n-step prediction analysis, prediction accuracy degrades over longer horizons. Techniques like observation re-encoding (using real observations periodically during imagination to reset the latent state) or training with longer sequences could mitigate this.

10. Reflection on the Project and Course

This project crystallized many of the concepts taught throughout CSC 580 into a single, cohesive system. Implementing Dreamer V1 from scratch required me to deeply understand not just the algorithm itself, but the broader landscape of ideas that support it.

The reinforcement learning foundations value functions, policy gradients, temporal difference learning, and the exploration-exploitation tradeoff are all present in the actor-critic component. The

critic uses TD-λ bootstrapping (a concept from the early lectures on value-based methods) while the actor uses policy gradient optimization through differentiable dynamics (connecting to the policy optimization lectures).

The world model concept itself bridges supervised and reinforcement learning. The RSSM's training is essentially supervised learning on sequences of environment transitions, while the actor-critic training is pure RL but conducted in a "simulated simulator." This duality highlights a key insight from the course: RL need not always involve direct environment interaction. By learning a model, we can transfer the sample efficiency of supervised learning to the RL setting.

The representation learning aspect learning a compact latent space that captures the environment's essential dynamics connects to broader ideas in deep learning about what makes a good representation. The fact that the encoder, dynamics model, reward predictor, and decoder are all trained jointly through complementary objectives is an elegant example of multi-task learning.
Working with the continuous action space brought home the practical challenges of extending RL beyond discrete settings. The tanh squashing, log-probability correction, and careful action range management are non-trivial engineering details that have significant impact on learning stability.
Working on this project deepened my understanding of Dreamer in particular and of reinforcement learning in general, and while there are many refinements possible, this project gave me a solid, hands-on grasp of model-based RL.

11. Files Generated

File	Description
Dream_the_Road_TinyDreamer.ipynb	Complete source code notebook with all implementations
tiny_dreamer_checkpoint.pt	Saved model weights and training metrics
videos/prediction_iter_*.mp4	Side-by-side prediction vs ground truth at training checkpoints
videos/prediction_comparison_final.mp4	Final n-step prediction comparison video
videos/prediction_grid.png	Grid of ground truth vs predicted frames
videos/agent_episode_*.mp4	Full driving episodes with the trained agent
videos/dream_sequence.mp4	Pure imagination rollout (no real env)
videos/dream_frames.png	Visualization of selected dream frames
videos/dream_reward_trajectory.png	Imagined reward over dream steps
videos/training_curves.png	Loss curves for all model components
videos/episode_rewards.png	Episode reward progression
videos/evaluation_metrics.png	Prediction MSE and reward accuracy plots

12. References

1. Hafner, D., Lillicrap, T., Ba, J., & Norouzi, M. (2020). *Dream to Control: Learning Behaviors by Latent Imagination.* International Conference on Learning Representations (ICLR 2020). arXiv:1912.01603.
2. Hafner, D., Lillicrap, T., Norouzi, M., & Ba, J. (2021). *Mastering Atari with Discrete World Models.* ICLR 2021. (Dreamer V2)
3. Hafner, D., Pasukonis, J., Ba, J., & Lillicrap, T. (2023). *Mastering Diverse Domains through World Models.* arXiv:2301.04104. (Dreamer V3)
4. Haarnoja, T., Zhou, A., Abbeel, P., & Levine, S. (2018). *Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor.* ICML 2018.
5. Cho, K., van Merrienboer, B., Gulechre, C., et al. (2014). *Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation.* EMNLP 2014. (GRU)
6. Leurent, E. (2018). *An Environment for Autonomous Driving Decision-Making.* GitHub: highway-env. https://github.com/Farama-Foundation/HighwayEnv